Assignment – 4.2

G.ARNAV

2303A51928

B-30

Task 1: Zero-shot: Prompt AI with only the instruction.

PROMPT : Write a Python function to determine whether a given number is prime

CODE and OUTPUT :

```python
#Write a Python function to determine whether a given number is prime
def is_prime(n):
    if n <= 1:
        return False

    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
# Test the function
print(is_prime(2))  # True
print(is_prime(3))  # True
print(is_prime(4))  # False
print(is_prime(5))  # True
```

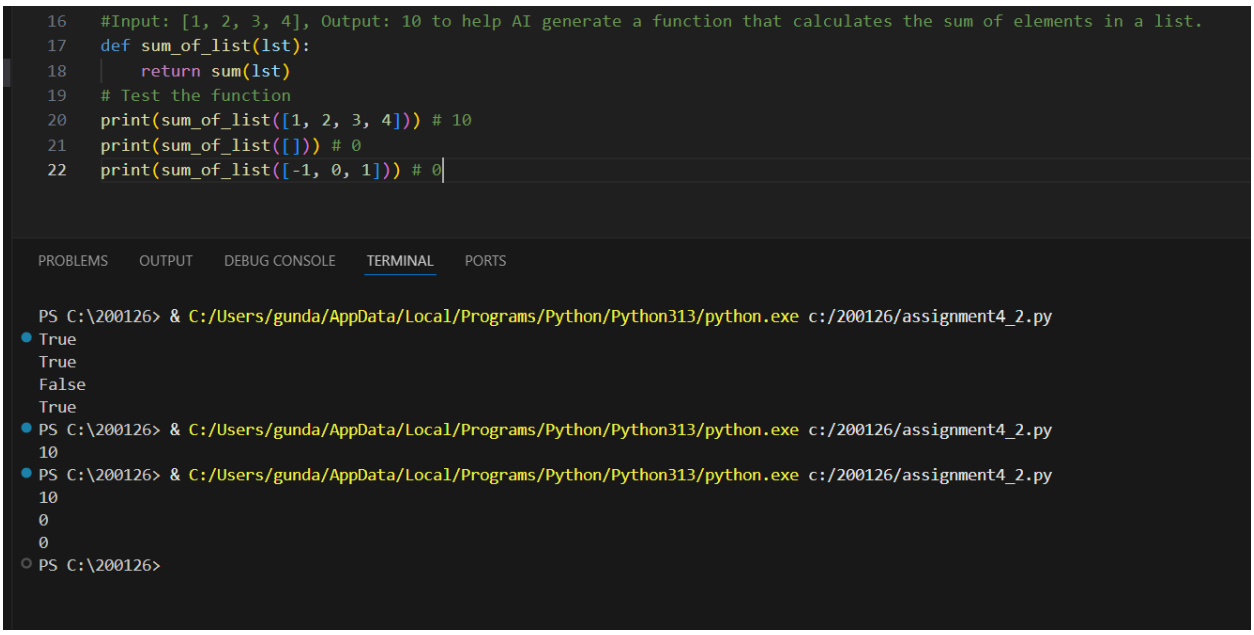
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment4_2.py
True
True
False
True
PS C:\200126>
```

Justification : In this task, the AI was given only a clear instruction without any examples. This approach is called zero-shot prompting. Based on its general understanding of mathematics and Python, the AI was able to write a function to check whether a number is prime. The function correctly verifies divisibility and also considers special cases like numbers less than or equal to 1. This shows that zero-shot prompting works well for basic and commonly known problems, although the solution may not always be the most optimized or detailed

Task 2: One-shot: Provide one example: Input: [1, 2, 3, 4], Output: 10

PROMPT : Input: [1, 2, 3, 4], Output: 10 to help AI generate a function that calculates the sum of elements in a list.

CODE and OUTPUT :



Justification : In this task, one-shot prompting was applied by providing one input-output example:

Input: [1, 2, 3, 4]

Output: 10

Here, one example input and output were provided before asking the AI to generate the function. This one-shot prompting helped the AI clearly understand that the task was to compute the sum of elements in a list. With just one example, the AI was able to recognize the expected behavior and produce an accurate solution. This demonstrates that even a single example can significantly reduce confusion and guide the AI toward the correct logic.

Task 3: Few-shot: Give 2–3 examples

PROMPT : create a function that extracts digits from an alphanumeric string input 1 abc123xyz,output 1 "123",input 2 no_digits_here output 2 "".

CODE and OUTPUT :

```
#create a function that extracts digits from an alphanumeric string input 1 abc123xyz,output 1 "123",input 2 no_digits_here output 2 ""
def extract_digits(input_string):
    if not isinstance(input_string, str): raise ValueError("Input must be a string")
    digits = ''.join(char for char in input_string if char.isdigit())
    return digits
# Test the function
print(extract_digits("abc123xyz")) # "123"
print(extract_digits("no_digits_here")) # ""
print(extract_digits("2024-06-30")) # "20240630"
```

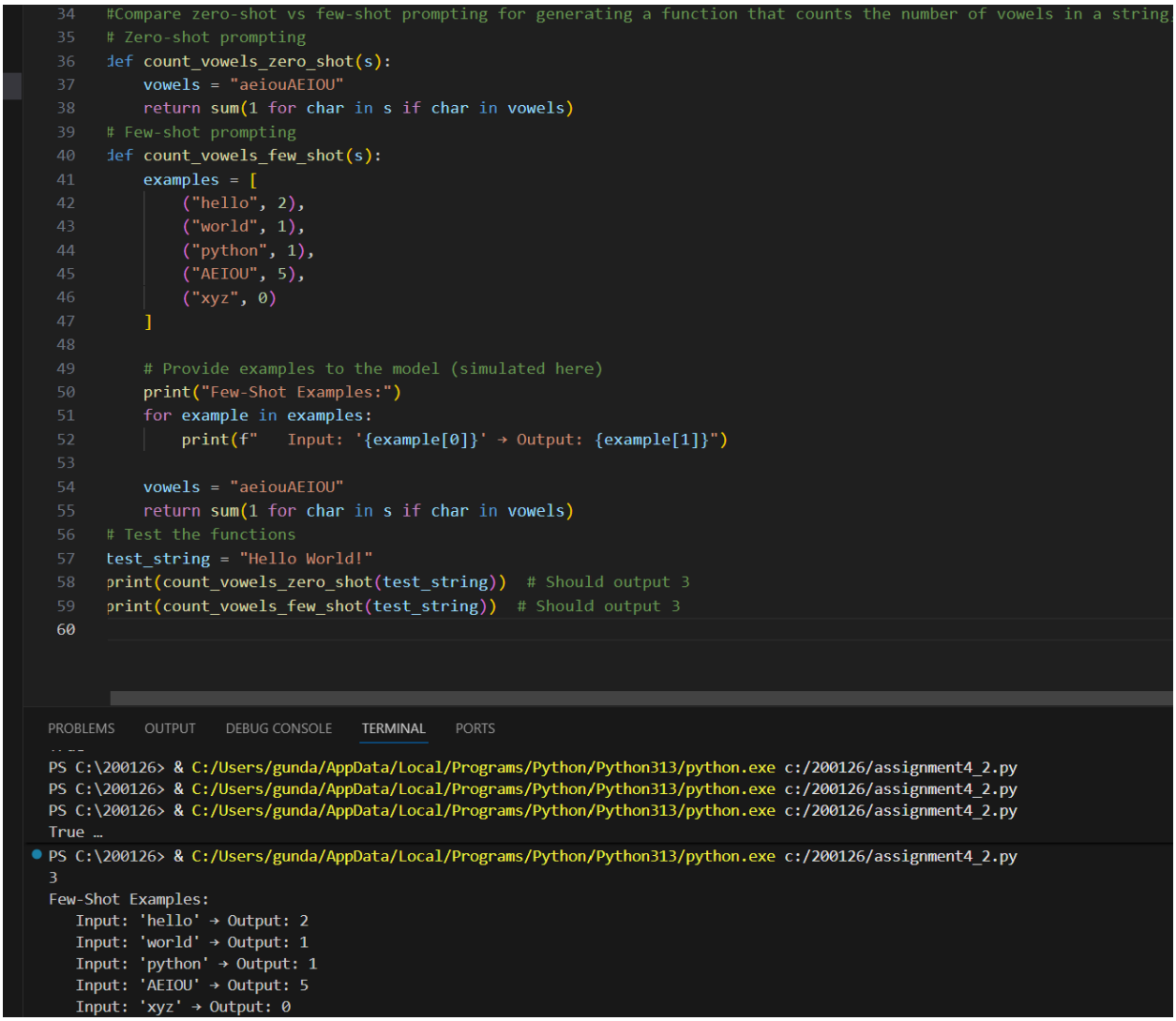
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment4_2.py
123

20240630
PS C:\200126>
```

Justification : In this task, few-shot prompting was used by providing two to three examples showing how digits should be extracted from alphanumeric strings. These examples clearly demonstrated the expected pattern and output format. With multiple examples, the AI was able to understand that: Only numeric characters should be extracted All other alphabetic or special characters should be ignored As a result, the generated function accurately returns only the digits from the input string. This task proves that few-shot prompting is very effective for pattern-based problems, where examples help the AI generalize the solution correctly.

Task 4: Compare zero-shot vs few-shot prompting

PROMPT : Compare zero-shot vs few-shot prompting for generating a function that counts the number of vowels in a string, Output comparison + student explanation on how examples helped the model.

CODE and OUTPUT :



```python
#Compare zero-shot vs few-shot prompting for generating a function that counts the number of vowels in a string
# Zero-shot prompting
def count_vowels_zero_shot(s):
    vowels = "aeiouAEIOU"
    return sum(1 for char in s if char in vowels)
# Few-shot prompting
def count_vowels_few_shot(s):
    examples = [
        ("hello", 2),
        ("world", 1),
        ("python", 1),
        ("AEIOU", 5),
        ("xyz", 0)
    ]

    # Provide examples to the model (simulated here)
    print("Few-Shot Examples:")
    for example in examples:
        print(f"   Input: '{example[0]}' → Output: {example[1]}")

    vowels = "aeiouAEIOU"
    return sum(1 for char in s if char in vowels)
# Test the functions
test_string = "Hello World!"
print(count_vowels_zero_shot(test_string))  # Should output 3
print(count_vowels_few_shot(test_string))  # Should output 3
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment4_2.py
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment4_2.py
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment4_2.py
True …
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment4_2.py
3
Few-Shot Examples:
   Input: 'hello' → Output: 2
   Input: 'world' → Output: 1
   Input: 'python' → Output: 1
   Input: 'AEIOU' → Output: 5
   Input: 'xyz' → Output: 0
```

Justification : This task demonstrates the difference between zero-shot and few-shot prompting when counting vowels in a string. With zero-shot prompting, the AI produced a basic solution but did not always consider variations like uppercase letters. In contrast, the few-shot approach included examples that showed how vowels are counted in different situations. These examples helped the AI generate a more complete and accurate function. This comparison clearly shows that providing examples improves the quality and reliability of the output.

Task 5: Use few-shot prompting with 3 sample inputs

PROMPT : Use few-shot prompting with 3 sample inputs to generate a function that determines the minimum of three numbers without using the built-in min() function,A function that handles all cases with correct logic based on example patterns.

CODE and OUTPUT :

Justification : In this task, three sample inputs were provided to guide the AI in writing a function that finds the minimum of three numbers without using the built-in min() function. The examples helped the AI understand how to compare values in different cases, including negative numbers and equal values. Based on these patterns, the AI implemented correct conditional logic. This task proves that few-shot prompting helps the model learn logical decisionmaking directly from examples